

Appunti sul progetto di Pneumonia Detection

Contents

1	ResNet	3
1.1	Bottleneck	3
1.1.1	Batch Normalization	4
1.2	Struttura di ResNet-50	4
1.2.1	Global Average Pooling	4
2	Feature Pyramid Network	5
2.1	Motivazione	5
2.2	Struttura della FPN	5
2.3	FPN utilizzata nel progetto	6
3	Detection Head	6
3.1	Idea generale	6
3.2	RetinaNet	7
3.3	FCOS	7
3.4	Detection head del paper sulla pneumonia	8
3.5	Detection head del progetto	8
3.6	Differenze rispetto ai paper	8
3.6.1	Group Normalization	9
4	Target Generator	9
4.1	Scopo	9
4.2	Target prodotti	9
4.3	Conversione delle location	10
4.4	Verifica della presenza della location nel box	10
4.5	Calcolo delle distanze LTRB	10
4.6	Assegnazione in base alla scala	11
4.7	Gestione di più bounding box compatibili	11
4.8	Calcolo della Center-ness	11
4.9	Costruzione finale dei target	12
5	Loss Function	12
5.1	Classification Loss: Focal Loss	12
5.2	Regression Loss: Generalized IoU	13
5.3	Centerness Loss	14
5.4	Normalizzazione della Loss	15
5.5	Confronto con i paper	15
5.5.1	Paper sulla pneumonia	15
5.5.2	FCOS	15
5.5.3	Progetto	16
5.5.4	Scelta di una formulazione FCOS-like	16
5.5.5	Feature-map Locations and Reference Boxes	17

6	TrainerV2	17
6.1	Scopo del TrainerV2	17
6.2	Freeze e Unfreeze del Backbone	18
6.3	Learning Rate Warmup	19
6.4	Cosine Annealing	19
6.5	Differential Learning Rate	20
6.6	Exponential Moving Average	21
6.7	Gradient Clipping	21
6.8	Training Loop	22
6.9	Validation	22
6.10	Checkpoint	23
6.11	Schema complessivo	23

1 ResNet

La ResNet (Residual Network) introduce le *residual connections* per facilitare l'addestramento di reti molto profonde. L'idea è che gli strati convoluzionali non debbano necessariamente apprendere direttamente la funzione desiderata $H(x)$, ma possano apprendere una funzione residua $F(x)$ rispetto all'input:

$$F(x) = H(x) - x$$

Il risultato del blocco viene quindi ottenuto come:

$$H(x) = F(x) + x$$

L'input x viene quindi mantenuto attraverso una *shortcut connection* e sommato all'output della trasformazione residua. Questa struttura facilita l'ottimizzazione delle reti profonde, poiché, nel caso in cui la trasformazione ottimale fosse vicina all'identità, il blocco deve imparare una correzione rispetto all'input invece dell'intera trasformazione. Il paper originale introduce questa formulazione come principio fondamentale della residual learning.

1.1 Bottleneck

ResNet-50 utilizza un particolare tipo di residual block chiamato *bottleneck block*. Il blocco è costituito da tre convoluzioni:

$$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$$

La prima convoluzione 1×1 riduce il numero di canali della rappresentazione. La convoluzione 3×3 , che è la principale convoluzione spaziale del blocco, opera quindi su un numero ridotto di canali, diminuendo il costo computazionale. Infine, la seconda convoluzione 1×1 aumenta nuovamente il numero di canali, riportandolo alla dimensione necessaria per effettuare la somma con la shortcut.

Ad esempio, un blocco può trasformare una rappresentazione:

$$256 \rightarrow 64 \rightarrow 64 \rightarrow 256$$

dove il passaggio a 64 canali costituisce il *bottleneck*.

La struttura completa del blocco può essere rappresentata come:

$$x \rightarrow 1 \times 1 \rightarrow BN \rightarrow ReLU \rightarrow 3 \times 3 \rightarrow BN \rightarrow ReLU \rightarrow 1 \times 1 \rightarrow BN$$

e, in parallelo, l'input viene trasmesso attraverso la shortcut. I due rami vengono poi sommati e viene applicata una ReLU finale:

$$y = ReLU(F(x) + x)$$

Quando le dimensioni dell'input e dell'output non coincidono, la shortcut viene trasformata tramite una convoluzione 1×1 per adattarne numero di canali e, se necessario, risoluzione spaziale.

Il bottleneck è stato introdotto nel paper originale proprio per rendere più efficiente la costruzione di reti molto profonde: le convoluzioni 1×1 riducono e poi aumentano le dimensioni, lasciando alla convoluzione 3×3 una rappresentazione più piccola su cui operare.

1.1.1 Batch Normalization

La *Batch Normalization* (BN) normalizza le attivazioni prodotte da una convoluzione utilizzando la media e la varianza calcolate sul mini-batch. Successivamente, le attivazioni normalizzate vengono trasformate tramite due parametri apprendibili, che permettono alla rete di adattarne nuovamente scala e traslazione.

In ResNet, la Batch Normalization viene applicata dopo ogni convoluzione e prima della funzione di attivazione ReLU. La sua funzione è rendere più stabile l'addestramento della rete e facilitare l'ottimizzazione dei numerosi strati convoluzionali.

1.2 Struttura di ResNet-50

ResNet-50 è costruita come una sequenza di quattro stage principali, ciascuno composto da più Bottleneck block. La configurazione di ResNet-50 utilizza rispettivamente:

$$[3, 4, 6, 3]$$

Bottleneck nei quattro stage `conv2_x`, `conv3_x`, `conv4_x` e `conv5_x`.

La struttura può essere riassunta come:

$$\text{Input} \rightarrow 7 \times 7 \text{ Conv} \rightarrow \text{MaxPool} \rightarrow \text{conv2_x} \rightarrow \text{conv3_x} \rightarrow \text{conv4_x} \rightarrow \text{conv5_x}$$

Per un input RGB di dimensione 224×224 , le principali dimensioni delle feature map sono:

$$224 \times 224 \rightarrow 112 \times 112 \rightarrow 56 \times 56 \rightarrow 28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$$

mentre il numero di canali aumenta progressivamente:

$$64 \rightarrow 256 \rightarrow 512 \rightarrow 1024 \rightarrow 2048.$$

Il primo layer utilizza una convoluzione 7×7 con stride 2, seguita da un max pooling con stride 2. I successivi cambiamenti di risoluzione avvengono all'inizio degli stage `conv3_x`, `conv4_x` e `conv5_x`, dove il primo Bottleneck utilizza stride 2. Nel codice, quindi, il primo Bottleneck di ogni nuovo stage è anche quello che adatta la dimensione della shortcut quando necessario.

1.2.1 Global Average Pooling

Al termine della rete, la ResNet originale utilizza un *Global Average Pooling* (GAP) per trasformare la feature map finale in un vettore compatto.

Nel caso di ResNet-50, la feature map finale ha dimensione:

$$7 \times 7 \times 2048.$$

Il Global Average Pooling calcola, per ogni canale, la media dei valori presenti su tutte le posizioni spaziali:

$$y_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W x_{c,i,j}.$$

La dimensione spaziale viene quindi ridotta da 7×7 a 1×1 , ottenendo:

$$7 \times 7 \times 2048 \rightarrow 1 \times 1 \times 2048.$$

Successivamente il tensore viene appiattito e passato al layer fully connected che produce le predizioni finali.

Il vantaggio del Global Average Pooling è che permette di riassumere l'intera mappa spaziale di ciascun canale in un singolo valore, evitando l'utilizzo di una grande quantità di parametri tipica di un fully connected applicato direttamente alla feature map. Il GAP non verrà usato nel caso in cui ResNet debba funzionare come backbone.

2 Feature Pyramid Network

2.1 Motivazione

Una rete convoluzionale profonda, come ResNet-50, produce una gerarchia di feature map a diverse risoluzioni spaziali. I layer più profondi hanno una risoluzione spaziale più bassa, ma contengono informazioni semantiche più ricche, mentre i layer più superficiali mantengono una maggiore risoluzione spaziale e una localizzazione più precisa, ma hanno informazioni semantiche meno forti.

Questo rappresenta un problema per l'object detection, perché gli oggetti possono avere dimensioni molto diverse. Le feature profonde sono utili per riconoscere *cosa* è presente nell'immagine, ma sono meno precise nel dire *dove* si trova. Al contrario, le feature superficiali sono più precise spazialmente, ma hanno una rappresentazione semantica più debole.

La Feature Pyramid Network (FPN) serve quindi a combinare queste due informazioni. L'idea è prendere le feature semantiche dei layer profondi e propagarle verso i layer a maggiore risoluzione, combinandole con le feature superficiali. In questo modo si ottengono feature map a diverse scale che mantengono sia una buona informazione semantica sia una buona precisione spaziale.¹

2.2 Struttura della FPN

La FPN originale utilizza le feature ottenute dagli ultimi blocchi residuali di ResNet, indicate come C_2 , C_3 , C_4 e C_5 . Queste feature hanno rispettivamente stride 4, 8, 16 e 32 rispetto all'immagine di input.

La costruzione della piramide parte dalla feature più profonda C_5 . Questa viene trasformata tramite una convoluzione 1×1 e costituisce il punto di partenza del percorso *top-down*. La feature viene poi sovracampionata di un fattore 2 e sommata alla corrispondente feature proveniente dal backbone, dopo che anch'essa è stata trasformata tramite una convoluzione 1×1 .

Il procedimento viene ripetuto progressivamente verso i livelli a maggiore risoluzione:

$$C_5 \rightarrow P_5$$

$$C_4 + \text{Upsample}(P_5) \rightarrow P_4$$

$$C_3 + \text{Upsample}(P_4) \rightarrow P_3$$

$$C_2 + \text{Upsample}(P_3) \rightarrow P_2$$

La convoluzione 1×1 viene utilizzata per portare le feature del backbone allo stesso numero di canali, mentre l'upsampling permette di allineare la risoluzione spaziale delle feature prima della somma.

Dopo la fusione viene applicata una convoluzione 3×3 per ottenere la feature finale di ciascun livello. Nel paper originale tutti i livelli della piramide hanno 256 canali.²

¹Tsung-Yi Lin et al., *Feature Pyramid Networks for Object Detection*, CVPR 2017.

²La FPN originale utilizza il percorso *top-down* con *lateral connections* e produce i livelli P_2 - P_5 .

2.3 FPN utilizzata nel progetto

Nel nostro progetto la FPN è leggermente diversa da quella originale, perché è stata adattata al detector utilizzato successivamente.

La ResNet-50 fornisce le feature:

$$C_3, C_4, C_5$$

con stride rispettivamente:

$$8, 16, 32.$$

A differenza della FPN originale, C_2 non viene utilizzata. La costruzione del percorso top-down rimane però la stessa:

$$C_5 \rightarrow P_5$$

$$C_4 + \text{Upsample}(P_5) \rightarrow P_4$$

$$C_3 + \text{Upsample}(P_4) \rightarrow P_3.$$

Anche in questo caso le feature provenienti dal backbone vengono prima proiettate tramite convoluzioni 1×1 a 256 canali. Dopo la somma viene applicata una convoluzione 3×3 .

La differenza principale è che il nostro modello non si ferma a P_3 , P_4 e P_5 , ma estende ulteriormente la piramide:

$$P_5 \rightarrow P_6 \rightarrow P_7.$$

P_6 viene ottenuta tramite una convoluzione 3×3 con stride 2 applicata a P_5 , mentre P_7 viene ottenuta nello stesso modo partendo da P_6 .

La piramide finale è quindi:

$$P_3, P_4, P_5, P_6, P_7$$

con stride:

$$8, 16, 32, 64, 128.$$

Questa scelta segue la struttura del detector descritto nel paper sulla pneumonia, che utilizza cinque livelli della feature pyramid con stride 8, 16, 32, 64 e 128.³

In sintesi, la FPN permette di avere feature a più scale, combinando informazioni semantiche profonde con informazioni spaziali più precise dei livelli superficiali. Nel nostro caso, questi livelli vengono poi utilizzati dal detector FCOS per effettuare le predizioni a scale differenti.

3 Detection Head

3.1 Idea generale

La *detection head* è il componente della rete che riceve le feature estratte dal backbone e dalla FPN e le trasforma nelle predizioni necessarie per l'object detection. In generale, una head può essere suddivisa in più branch, ognuna specializzata in un compito differente, ad esempio la classificazione dell'oggetto e la regressione della sua bounding box.

³Linghua Wu et al., *Pneumonia detection based on RSNA dataset and anchor-free deep learning detector*, Scientific Reports, 2024.

L'idea generale è quindi:

$$\text{Feature Map} \rightarrow \text{Detection Head} \rightarrow \begin{cases} \text{Classification} \\ \text{Localization} \end{cases}$$

Nel caso dei detector anchor-free, la predizione viene effettuata direttamente su ogni posizione spaziale della feature map invece di utilizzare bounding box predefinite come riferimento.

3.2 RetinaNet

Un esempio importante è RetinaNet. La rete utilizza due subnet separate, una per la classificazione e una per la regressione delle bounding box. Entrambe ricevono una feature map della FPN e sono costituite da più convoluzioni 3×3 , seguite da un layer finale di predizione.

La classification subnet produce, per ogni posizione spaziale, le probabilità relative alle classi e agli anchor presenti in quella posizione, mentre la box regression subnet produce i parametri necessari per correggere le coordinate degli anchor.⁴

La struttura può essere schematizzata come:

$$\text{FPN feature} \rightarrow \begin{cases} \text{Classification subnet} \\ \text{Regression subnet} \end{cases}$$

In RetinaNet le due subnet hanno la stessa struttura generale, ma utilizzano parametri separati. La classification subnet termina con KA output per posizione, mentre la regression subnet termina con $4A$ output, dove K è il numero di classi e A il numero di anchor per posizione.

3.3 FCOS

FCOS mantiene la separazione tra classificazione e regressione, ma elimina il riferimento agli anchor. Ogni posizione della feature map viene trattata come una possibile location dell'oggetto e la regressione predice direttamente le distanze dalla posizione ai quattro lati della bounding box:

$$(l, t, r, b).$$

La detection head di FCOS contiene quindi una branch per la classificazione e una branch per la regressione. Il paper descrive per entrambe una sequenza di convolutional layers applicata alle feature della FPN. In aggiunta viene introdotta una terza branch, la *center-ness branch*, che predice quanto una location sia vicina al centro dell'oggetto e serve a ridurre l'influenza delle predizioni di bassa qualità.⁵

Lo schema concettuale è quindi:

$$\text{FPN feature} \rightarrow \begin{cases} \text{Classification} \\ \text{Regression} \\ \text{Center-ness} \end{cases}$$

La classificazione determina *che cosa* è presente in una determinata location, la regressione determina *dove* si trova la bounding box, mentre la center-ness fornisce un'indicazione della qualità della location utilizzata per effettuare la predizione.

⁴Tsung-Yi Lin et al., *Focal Loss for Dense Object Detection*, ICCV 2017.

⁵Zhi Tian et al., *FCOS: Fully Convolutional One-Stage Object Detection*, ICCV 2019.

3.4 Detection head del paper sulla pneumonia

Il paper utilizzato come riferimento per questo progetto descrive una head più semplice. Ogni livello della feature pyramid viene elaborato inizialmente da una convoluzione 3×3 e successivamente da due convoluzioni 1×1 che producono due predizioni distinte, indicate come *CENTER* e *SCALE*.

$$\text{FPN feature} \rightarrow 3 \times 3 \rightarrow \begin{cases} 1 \times 1 \rightarrow \text{CENTER} \\ 1 \times 1 \rightarrow \text{SCALE} \end{cases}$$

Il branch CENTER viene utilizzato per individuare le location associate al centro degli oggetti, mentre il branch SCALE viene utilizzato per predire la scala della bounding box.⁶

3.5 Detection head del progetto

La detection head utilizzata nel progetto è più vicina a quella di FCOS che a quella descritta nel paper sulla pneumonia.

Ogni livello della FPN $P_3, \dots, P_7$ viene passato a una *DetectionHead*. La head è divisa in due tower principali:

$$\text{Feature} \rightarrow \begin{cases} \text{Classification tower} \\ \text{Regression tower} \end{cases}$$

La classification tower è costituita, nel progetto, da:

$$3 \times 3 \text{ Conv} \rightarrow \text{GroupNorm} \rightarrow \text{ReLU} \rightarrow 1 \times 1 \text{ Conv}.$$

La regression tower utilizza una struttura analoga:

$$3 \times 3 \text{ Conv} \rightarrow \text{GroupNorm} \rightarrow \text{ReLU}.$$

Dalla regression tower vengono poi prodotte due predizioni separate:

$$\text{Regression tower} \rightarrow \begin{cases} 1 \times 1 \rightarrow (l, t, r, b) \\ 1 \times 1 \rightarrow \text{centerness} \end{cases}$$

Pertanto, per ogni location di ogni livello della FPN, il modello produce:

$$\text{classification}, \quad (l, t, r, b), \quad \text{centerness}.$$

Nel progetto la classificazione produce attualmente un solo valore per location, perché viene utilizzata una sola classe target, la pneumonia.

3.6 Differenze rispetto ai paper

La struttura del progetto non coincide esattamente con nessuno dei due paper.

Rispetto al paper sulla pneumonia, il progetto aggiunge una *classification branch* esplicita e utilizza una regressione *LTRB*, più vicina alla formulazione FCOS, invece della distinzione CENTER/SCALE descritta nel paper.

Rispetto a FCOS, la struttura è semplificata: il paper originale utilizza quattro convolutional layers nelle branch di classificazione e regressione, mentre il progetto utilizza una sola convoluzione 3×3 per ciascuna tower, seguita da Group Normalization e ReLU.

⁶Linghua Wu et al., *Pneumonia detection based on RSNA dataset and anchor-free deep learning detector*, Scientific Reports, 2024.

Inoltre, FCOS condivide le head tra i diversi livelli della feature pyramid, mentre nel progetto vengono create cinque istanze separate di **DetectionHead**, una per ciascun livello $P_3, \dots, P_7$. Il paper FCOS descrive infatti la condivisione delle head tra i diversi livelli, pur introducendo modifiche specifiche per gestire i differenti range di scala.

Nel complesso, quindi, la detection head del progetto può essere considerata una *FCOS-like detection head*: ne mantiene la formulazione anchor-free, la classificazione per location, la regressione *LTRB* e la center-ness branch, ma utilizza una struttura più semplice e alcune scelte architetturali specifiche del progetto.

3.6.1 Group Normalization

La *Group Normalization* (GN) è una tecnica di normalizzazione delle feature map che divide i canali in gruppi e normalizza i valori all'interno di ciascun gruppo.

A differenza della Batch Normalization, la Group Normalization non utilizza le statistiche dell'intero batch. Questo la rende meno dipendente dalla dimensione del batch e quindi particolarmente utile quando si lavora con batch molto piccoli, situazione comune nell'object detection a causa dell'elevato costo computazionale delle immagini.

Nel progetto la Group Normalization viene utilizzata all'interno delle classification e regression tower della detection head:

$$\text{Conv} \rightarrow \text{GroupNorm} \rightarrow \text{ReLU}.$$

Il suo scopo è stabilizzare le attivazioni e rendere l'addestramento più robusto, senza richiedere batch grandi.

4 Target Generator

4.1 Scopo

Il *Target Generator* è un componente utilizzato durante il training per trasformare le bounding box del ground truth in target compatibili con la formulazione FCOS.

La Detection Head produce infatti una predizione per ogni posizione spaziale delle feature map della FPN, ma durante l'addestramento è necessario stabilire che cosa ogni posizione dovrebbe predire. Il Target Generator esegue proprio questa associazione.

La pipeline può essere vista come:

$$\text{Ground Truth} \rightarrow \text{Target Generator} \rightarrow \text{Target} \rightarrow \text{Loss}$$

Il Target Generator non fa parte del modello neurale e viene utilizzato solamente durante il training. Durante l'inference il ground truth non è disponibile e quindi questo componente non viene utilizzato.

4.2 Target prodotti

Per ogni livello della FPN, il Target Generator produce tre target:

- **positive**: indica se una location della feature map corrisponde a un oggetto;
- **ltrb**: contiene le quattro distanze dalla location ai lati della bounding box;
- **centerness**: indica quanto la location si trova vicino al centro della bounding box.

In questo modo, per ogni location positiva, il modello deve imparare a predire:

$$(l, t, r, b)$$

e il relativo valore di *centerness*.

4.3 Conversione delle location

La prima operazione consiste nel trasformare una posizione (x, y) della feature map nelle corrispondenti coordinate dell'immagine originale.

Per una feature map con stride s , il centro della location viene calcolato come:

$$x_{\text{img}} = (x + 0.5)s, \quad y_{\text{img}} = (y + 0.5)s.$$

Il termine 0.5 serve a considerare il centro della cella della feature map.

Questa operazione è necessaria perché le bounding box del ground truth sono espresse nelle coordinate dell'immagine originale, mentre le predizioni della Detection Head sono associate alle location delle feature map.

4.4 Verifica della presenza della location nel box

Per ogni location viene verificato se il punto trasformato nelle coordinate dell'immagine cade all'interno di una bounding box.

Se:

$$x_1 \leq x \leq x_2$$

e

$$y_1 \leq y \leq y_2,$$

la location è considerata appartenente al bounding box e può quindi diventare un candidato positivo.

Se la location si trova fuori dal box, quel ground truth non può essere assegnato alla location.

4.5 Calcolo delle distanze LTRB

Quando una location si trova all'interno di una bounding box, vengono calcolate le distanze tra la location e i quattro lati del box:

$$l = x - x_1$$

$$t = y - y_1$$

$$r = x_2 - x$$

$$b = y_2 - y.$$

Il vettore risultante è:

$$(l, t, r, b).$$

Questa è la rappresentazione utilizzata da FCOS per la regressione della bounding box: invece di predire direttamente (x_1, y_1, x_2, y_2) , il modello predice la distanza della location dai quattro lati del box.⁷

⁷Zhi Tian et al., *FCOS: Fully Convolutional One-Stage Object Detection*, ICCV 2019.

4.6 Assegnazione in base alla scala

Non tutte le feature map sono adatte a rappresentare oggetti della stessa dimensione. Per questo il Target Generator utilizza un diverso *regression range* per ogni livello della FPN.

Nel progetto:

Stride	Regression range
8	[0, 64)
16	[64, 128)
32	[128, 256)
64	[256, 512)
128	[512, ∞)

Per ogni location viene considerata la massima delle quattro distanze:

$$m = \max(l, t, r, b).$$

Il bounding box può essere assegnato a un determinato livello solo se m rientra nel range associato a quel livello.

Questo meccanismo permette di distribuire gli oggetti di dimensioni diverse tra i diversi livelli della FPN e riduce l'ambiguità tra le feature map. L'uso di un diverso intervallo di regressione per ogni livello è una parte fondamentale della formulazione FCOS.⁸

4.7 Gestione di più bounding box compatibili

Una location può trovarsi contemporaneamente all'interno di più bounding box. In questo caso può esserci ambiguità su quale oggetto debba essere utilizzato come target.

Il progetto risolve questa situazione scegliendo il bounding box con area minore tra quelli compatibili.

$$B^* = \arg \min_{B_i} \text{Area}(B_i)$$

Questa strategia segue la regola utilizzata nella formulazione FCOS per risolvere le location appartenenti a più ground-truth box.⁹

4.8 Calcolo della Center-ness

Una volta scelto il bounding box, viene calcolato il target di *centerness* a partire dalle quattro distanze LTRB:

$$\text{centerness} = \sqrt{\frac{\min(l, r)}{\max(l, r)} \cdot \frac{\min(t, b)}{\max(t, b)}}.$$

Una location vicina al centro del bounding box avrà un valore vicino a 1, mentre una location vicina ai bordi avrà un valore più basso.

Il motivo è che le location lontane dal centro tendono a produrre bounding box di qualità inferiore. FCOS introduce quindi la center-ness come una predizione separata utilizzata per ridurre il peso di queste detection durante l'inference.¹⁰

⁸Zhi Tian et al., *FCOS: Fully Convolutional One-Stage Object Detection*, ICCV 2019.

⁹Zhi Tian et al., *FCOS: Fully Convolutional One-Stage Object Detection*, ICCV 2019.

¹⁰Zhi Tian et al., *FCOS: Fully Convolutional One-Stage Object Detection*, ICCV 2019.

4.9 Costruzione finale dei target

Dopo aver eseguito tutte le verifiche, ogni location della feature map viene classificata come positiva o negativa.

Per una location negativa:

$$\text{positive} = 0$$

e i target di regressione e center-ness vengono impostati a zero.

Per una location positiva:

$$\text{positive} = 1$$

e vengono memorizzati:

$$\text{ltrb} = (l, t, r, b)$$

e

$$\text{centerness} \in [0, 1].$$

Il risultato finale è quindi una struttura di target della stessa organizzazione spaziale della feature map, che verrà successivamente confrontata con le predizioni della Detection Head attraverso la funzione di loss.

5 Loss Function

Durante il training, la *Detection Head* produce tre tipi di predizioni:

$$\text{classification}, \quad (l, t, r, b), \quad \text{centerness}.$$

Il *Target Generator* produce gli stessi tipi di target a partire dalle bounding box del ground truth. La funzione di loss misura quindi quanto le predizioni della rete differiscono dai target.

Nel progetto la loss totale è definita come combinazione di tre componenti:

$$L_{\text{total}} = \lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{reg}} L_{\text{reg}} + \lambda_{\text{ctr}} L_{\text{ctr}}.$$

Nel caso attuale tutti i pesi sono impostati a 1, quindi le tre componenti contribuiscono in ugual misura alla loss totale.

5.1 Classification Loss: Focal Loss

La classification branch deve determinare, per ogni location della feature map, se è presente un oggetto oppure se la location appartiene al background. Nel nostro caso è presente una sola classe, quindi il target è binario:

$$y = \begin{cases} 1 & \text{location positiva} \\ 0 & \text{background.} \end{cases}$$

Per questo motivo viene utilizzata una *sigmoid focal loss*.

La Focal Loss è stata introdotta da Lin et al. per affrontare il forte sbilanciamento tra esempi positivi e negativi tipico dei *dense detectors*.¹¹

La prima parte è la Binary Cross Entropy:

¹¹Tsung-Yi Lin et al., *Focal Loss for Dense Object Detection*, ICCV 2017.

$$BCE(p, y) = -[y \log(p) + (1 - y) \log(1 - p)],$$

dove p è la probabilità predetta dalla rete.

La Focal Loss introduce poi un fattore che riduce il contributo degli esempi facili:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t).$$

Dove p_t rappresenta la probabilità assegnata alla classe corretta:

$$p_t = \begin{cases} p & \text{se } y = 1 \\ 1 - p & \text{se } y = 0. \end{cases}$$

Il termine

$$(1 - p_t)^\gamma$$

è il termine fondamentale della Focal Loss. Se un esempio è classificato correttamente con alta confidenza, allora p_t è vicino a 1 e il termine diventa molto piccolo. In questo modo gli esempi facili contribuiscono poco alla loss, mentre gli esempi difficili ricevono un peso maggiore.

Il parametro α_t permette invece di bilanciare il contributo delle classi positive e negative.

Nel progetto vengono utilizzati:

$$\alpha = 0.25, \quad \gamma = 2.$$

La Focal Loss viene utilizzata sulla classification branch per tutte le location della feature map.

5.2 Regression Loss: Generalized IoU

La regression branch ha il compito di localizzare la bounding box. Per ogni location positiva il modello predice:

$$(l, t, r, b),$$

ovvero le distanze della location dai quattro lati della bounding box.

Nel progetto la predizione viene prima trasformata in una bounding box $XYXY$ e successivamente confrontata con la bounding box ground truth tramite la *Generalized Intersection over Union* (GIoU).

La IoU tra due bounding box A e B è:

$$IoU = \frac{|A \cap B|}{|A \cup B|}.$$

Il valore è compreso tra 0 e 1 e misura quanto le due bounding box si sovrappongono.

La sola IoU presenta però un problema quando le bounding box non si sovrappongono: in quel caso la IoU è 0 e non fornisce informazioni sulla distanza tra i due box.

La Generalized IoU introduce quindi il più piccolo bounding box C che contiene entrambe le box:

$$GIoU = IoU - \frac{|C \setminus (A \cup B)|}{|C|}.$$

Il secondo termine penalizza le situazioni in cui le due bounding box sono lontane tra loro.

La loss utilizzata è:

$$L_{GIoU} = 1 - GIoU.$$

Di conseguenza, quando la box predetta coincide con il ground truth:

$$GIoU \rightarrow 1 \quad \Rightarrow \quad L_{GIoU} \rightarrow 0.$$

Nel nostro codice la regression loss viene calcolata solo sulle location positive.

È importante notare che il modello non confronta direttamente i quattro valori *LTRB* con una loss L1. La procedura è:

$$LTRB \text{ prediction} \rightarrow \text{decode} \rightarrow XYXY \text{ prediction} \rightarrow GIoU \text{ loss}.$$

Questa scelta differisce dal paper sulla pneumonia, che utilizza una *Smooth L1 loss* per la regressione.¹²

5.3 Centerness Loss

La *centerness branch* deve imparare quanto una location sia vicina al centro della bounding box associata.

Il Target Generator calcola il target di centerness come:

$$c^* = \sqrt{\frac{\min(l, r)}{\max(l, r)} \cdot \frac{\min(t, b)}{\max(t, b)}}.$$

Il rapporto

$$\frac{\min(l, r)}{\max(l, r)}$$

misura quanto la location sia centrata orizzontalmente, mentre

$$\frac{\min(t, b)}{\max(t, b)}$$

misura quanto sia centrata verticalmente.

La radice quadrata combina i due valori.

Una location esattamente al centro del box tende ad avere:

$$c^* \approx 1,$$

mentre una location molto vicina a uno dei bordi tende ad avere:

$$c^* \approx 0.$$

Questo target viene utilizzato per insegnare alla centerness branch a distinguere le location che producono box di buona qualità da quelle lontane dal centro.

Nel progetto la centerness prediction viene confrontata con il target tramite la stessa *sigmoid focal loss* utilizzata per la classificazione:

$$L_{ctr} = -\alpha_t(1 - p_t)^\gamma \log(p_t).$$

La loss di centerness viene però calcolata solamente sulle location positive.

¹²Linghua Wu et al., *Pneumonia detection based on RSNA dataset and anchor-free deep learning detector*, Scientific Reports, 2024.

5.4 Normalizzazione della Loss

Le tre componenti vengono inizialmente sommate su tutte le immagini e tutti i livelli della FPN. Successivamente la loss viene normalizzata rispetto al numero totale di location positive:

$$N_{\text{pos}} = \sum_{\text{images}} \sum_{\text{levels}} \#\{\text{positive locations}\}.$$

La loss finale diventa quindi:

$$L_{\text{total}} = \frac{\lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{reg}} L_{\text{reg}} + \lambda_{\text{ctr}} L_{\text{ctr}}}{\max(1, N_{\text{pos}})}.$$

La normalizzazione impedisce che la scala della loss dipenda semplicemente dal numero di location positive presenti nel batch.

5.5 Confronto con i paper

La loss del progetto combina idee provenienti principalmente dalla famiglia FCOS e dalla Focal Loss, ma non coincide esattamente con la formulazione di uno dei due paper di riferimento.

5.5.1 Paper sulla pneumonia

Il paper sulla pneumonia utilizza due componenti principali:

$$L = L_{\text{center}} + L_{\text{scale}}.$$

Il CENTER viene addestrato tramite Focal Loss, mentre la previsione della scala viene addestrata tramite Smooth L1.¹³

La formulazione è quindi:

$$\text{CENTER} \rightarrow \text{Focal Loss}$$

$$\text{SCALE} \rightarrow \text{Smooth L1}.$$

5.5.2 FCOS

FCOS utilizza invece una formulazione basata su tre predizioni:

$$\text{classification}, \quad \text{regression}, \quad \text{centerness}.$$

La classification branch utilizza Focal Loss, la regressione utilizza una loss basata sulla IoU e la centerness utilizza una Binary Cross Entropy. Inoltre, la center-ness viene introdotta specificamente per ridurre il contributo delle prediction generate da location lontane dal centro degli oggetti.¹⁴

¹³Linghua Wu et al., *Pneumonia detection based on RSNA dataset and anchor-free deep learning detector*, Scientific Reports, 2024.

¹⁴Zhi Tian et al., *FCOS: Fully Convolutional One-Stage Object Detection*, ICCV 2019.

5.5.3 Progetto

Il progetto è quindi concettualmente più vicino a FCOS:

$$\boxed{\text{Classification} + \text{LTRB Regression} + \text{Centerness}}$$

ma presenta alcune modifiche:

$$\text{Classification} \rightarrow \text{Focal Loss}$$

$$\text{Regression} \rightarrow \text{GIoU Loss}$$

$$\text{Centerness} \rightarrow \text{Focal Loss.}$$

La differenza più evidente rispetto a FCOS è quindi la loss della centerness: FCOS utilizza Binary Cross Entropy, mentre nel progetto viene utilizzata la Focal Loss.

Rispetto al paper sulla pneumonia, invece, la differenza è maggiore, perché il progetto mantiene una classification branch esplicita, utilizza la regressione *LTRB* tipica della formulazione FCOS e sostituisce la Smooth L1 con la GIoU loss.

Nel complesso, la loss implementata nel progetto può quindi essere considerata una formulazione *FCOS-like*, adattata alle esigenze specifiche del progetto.

5.5.4 Scelta di una formulazione FCOS-like

Sebbene il progetto sia nato con l'obiettivo di riprodurre il detector descritto nel paper sulla pneumonia, durante l'implementazione alcune difficoltà di ottimizzazione hanno motivato l'adozione di una formulazione più vicina a FCOS.

In particolare, nelle prime versioni del modello il training risultava instabile, con episodi di gradient explosion. Anche una formulazione più vicina a FCOS mostrava inizialmente problemi di stabilità. L'analisi delle architetture e delle tecniche utilizzate nei detector ha portato all'impiego della *Group Normalization* nelle detection heads. Questa scelta è coerente con il paper FCOS, che utilizza Group Normalization nelle convolutional layers delle heads proprio per rendere il training più stabile.¹⁵

La formulazione finale del progetto è quindi maggiormente vicina a FCOS rispetto al detector descritto nel paper sulla pneumonia. Vengono infatti mantenuti diversi elementi caratteristici di FCOS: la predizione per location, la regressione *LTRB*, la classification branch, la center-ness branch, i regression ranges associati ai livelli della FPN e la struttura P_3-P_7 .

Il progetto non costituisce tuttavia una riproduzione esatta di FCOS. La detection head è più semplice, poiché utilizza una sola convoluzione 3×3 per ciascuna tower invece delle quattro convoluzioni previste dalla formulazione originale. Inoltre, le detection heads non condividono i parametri tra i diversi livelli della FPN.

Anche le funzioni di loss presentano alcune differenze. La regressione utilizza la Generalized IoU loss, coerente con una delle successive migliorie introdotte nel lavoro FCOS, mentre la center-ness viene addestrata tramite Focal Loss anziché Binary Cross Entropy.

Il modello può quindi essere considerato una variante semplificata e adattata al problema di pneumonia detection di un detector *FCOS-like*.

¹⁵Zhi Tian et al., *FCOS: Fully Convolutional One-Stage Object Detection*, ICCV 2019.

5.5.5 Feature-map Locations and Reference Boxes

In FCOS, ogni posizione spaziale della feature map rappresenta una *location* sulla quale il modello può effettuare una predizione. La location viene riportata nelle coordinate dell'immagine originale utilizzando lo stride del livello della FPN:

$$x_{\text{img}} = (x + 0.5)s, \quad y_{\text{img}} = (y + 0.5)s,$$

dove (x, y) è la posizione sulla feature map e s è lo stride del livello.

Ad esempio, una location di P_3 con stride 8 corrisponde ad un punto dell'immagine originale distante 8 pixel dalla location adiacente. In termini intuitivi, una singola location può quindi essere associata ad una cella di circa 8×8 pixel nell'immagine originale, anche se la feature in quella posizione può contenere informazioni provenienti da una regione molto più ampia a causa del *receptive field* della rete.

Nel FCOS originale la location viene concettualmente trattata come un punto rispetto al quale vengono predette direttamente le quattro distanze

$$(l, t, r, b).$$

Nel nostro progetto, invece, per eseguire il decoding viene utilizzato il `BoxLinearCoder` di `torchvision`. Questo componente utilizza un *reference box* anziché un semplice punto. Per questo motivo, a partire dalla location nell'immagine viene costruito un piccolo box centrato sulla location e con dimensione pari allo stride:

$$B_{\text{ref}} = \left[x - \frac{s}{2}, y - \frac{s}{2}, x + \frac{s}{2}, y + \frac{s}{2} \right].$$

Il reference box non rappresenta un'ipotesi sulla dimensione dell'oggetto e non costituisce un anchor box nel senso tradizionale dei detector anchor-based. È solamente una rappresentazione della location utilizzata come riferimento dal `BoxLinearCoder`.

La pipeline di decoding diventa quindi:

feature-map location $\rightarrow$ image coordinates $\rightarrow$ reference box $\rightarrow$ LTRB prediction $\rightarrow$ BoxLinearCoder $\rightarrow$ YYYY

6 TrainerV2

6.1 Scopo del TrainerV2

Il `TrainerV2` è una versione avanzata del trainer base del progetto. Non modifica l'architettura del detector, il *Target Generator*, la funzione di loss o il processo di post-processing, ma modifica principalmente la strategia utilizzata per ottimizzare i pesi del modello.

La classe eredita direttamente da `Trainer`:

```
class TrainerV2(Trainer):
```

e riutilizza quindi la maggior parte della logica già presente nel trainer base, come la gestione dei dataset, la costruzione dei target, il calcolo della loss, il gradient clipping, la validazione, il logging e i checkpoint.

Le principali modifiche introdotte da `TrainerV2` riguardano l'ottimizzazione:

- iniziale congelamento della ResNet;
- *learning rate warmup*;
- *cosine annealing*;

- learning rate differente per backbone e detection layers;
- *Exponential Moving Average* (EMA) dei pesi;
- gradient clipping.

La struttura generale rimane quindi:

input → model → prediction → target → loss → backpropagation → optimizer.

Le modifiche del TrainerV2 riguardano soprattutto il modo in cui l'optimizer aggiorna i parametri del modello nel tempo.

6.2 Freeze e Unfreeze del Backbone

Il backbone del detector è costituito dalla ResNet-50. Nel progetto è possibile mantenerlo congelato per un certo numero di epoche iniziali:

ResNet frozen

Durante questa fase i parametri della ResNet hanno:

```
requires_grad = False
```

e quindi non vengono aggiornati durante il backpropagation.

Al contrario, FPN e detection heads rimangono allenabili.

La logica è:

ResNet → frozen

FPN + Heads → trainable.

L'idea è sfruttare inizialmente le feature già apprese dal backbone pretrained e concentrarsi sull'apprendimento delle parti specifiche del detector.

Dopo il numero di epoche specificato, la ResNet viene sbloccata:

ResNet frozen → ResNet trainable.

Nel progetto, quando la ResNet viene congelata, viene anche mantenuta in modalità evaluation:

```
self.backbone.eval()
```

in modo da evitare che le statistiche interne della Batch Normalization cambino durante la fase di congelamento.

Il congelamento del backbone viene quindi utilizzato come una forma di *staged training*: inizialmente si adattano le parti nuove della rete e successivamente si permette anche al backbone di adattarsi al nuovo dominio.

6.3 Learning Rate Warmup

Il *warmup* è una tecnica utilizzata per evitare di iniziare il training con un learning rate troppo elevato.

Il learning rate determina la dimensione degli aggiornamenti applicati ai parametri:

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} L.$$

Se il learning rate η_t è troppo grande nelle prime iterazioni, gli aggiornamenti possono essere troppo aggressivi e rendere il training instabile.

Nel TrainerV2 il learning rate viene quindi inizialmente impostato a un valore molto basso e aumentato progressivamente fino al learning rate base.

Nel codice viene utilizzato un **LinearLR**, con:

start factor = 0.01

e:

end factor = 1.

Con un learning rate base η_0 , durante il warmup il learning rate cresce quindi progressivamente da:

$0.01\eta_0$

fino a:

η_0 .

Il processo può essere rappresentato come:

small LR $\rightarrow$ progressive increase $\rightarrow$ base LR.

Nel progetto il warmup dura due epoche per impostazione predefinita.

L'obiettivo non è migliorare direttamente la capacità del modello, ma rendere più controllati i primi aggiornamenti dei pesi.

6.4 Cosine Annealing

Dopo il warmup, il TrainerV2 utilizza un *Cosine Annealing Learning Rate Scheduler*.

L'idea è far diminuire gradualmente il learning rate durante il training secondo una funzione coseno.

Una formulazione semplificata è:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t}{T} \pi \right) \right).$$

Dove:

- $\eta_{\max}$ è il learning rate iniziale;
- $\eta_{\min}$ è il learning rate minimo;
- t è il punto corrente del training;
- T è la durata del ciclo.

All'inizio:

$$\eta_t \approx \eta_{\max},$$

mentre verso la fine:

$$\eta_t \rightarrow \eta_{\min}.$$

L'idea intuitiva è utilizzare aggiornamenti più grandi quando il modello è ancora lontano da una buona soluzione e ridurre progressivamente la loro dimensione quando il training si avvicina alla regione di convergenza.

Nel TrainerV2 il warmup e il cosine annealing sono concatenati:

Warmup $\rightarrow$ Cosine Annealing.

Il learning rate segue quindi inizialmente una crescita controllata e successivamente una decrescita graduale.

6.5 Differential Learning Rate

Quando la ResNet viene sbloccata, il TrainerV2 non utilizza necessariamente lo stesso learning rate per tutte le parti del modello.

I parametri vengono infatti divisi in due gruppi:

Backbone

e

FPN + Detection Heads.

Nel progetto il learning rate del backbone è impostato a una frazione di quello utilizzato dalle parti nuove:

$$\eta_{\text{backbone}} = 0.1\eta_{\text{base}}.$$

Quindi, con:

$$\eta_{\text{base}} = 10^{-4},$$

si ottiene:

$$\eta_{\text{backbone}} = 10^{-5}$$

mentre FPN e detection heads continuano a utilizzare:

$$\eta_{\text{head}} = 10^{-4}.$$

La motivazione è che la ResNet è già pretrained e contiene feature general-purpose, mentre FPN e detection heads sono state introdotte per il problema specifico.

È quindi ragionevole voler modificare il backbone più lentamente rispetto alle parti nuove:

pretrained features $\rightarrow$ small updates

new detector layers $\rightarrow$ larger updates.

Questa strategia viene spesso indicata come *differential learning rate*.

6.6 Exponential Moving Average

Il TrainerV2 mantiene inoltre una seconda rappresentazione dei parametri del modello tramite una *Exponential Moving Average* (EMA).

L'idea è non utilizzare esclusivamente i pesi del modello nell'ultimo optimization step, ma mantenere una media mobile dei pesi osservati durante il training.

Se θ_t sono i parametri correnti e θ_t^{EMA} i parametri mediati, l'aggiornamento è:

$$\theta_t^{EMA} = \alpha \theta_{t-1}^{EMA} + (1 - \alpha) \theta_t.$$

Dove α è il fattore di decadimento.

Nel progetto:

$$\alpha = 0.999.$$

Un valore elevato significa che la EMA cambia lentamente e quindi tende a smussare le variazioni rapide dei parametri.

In forma intuitiva:

EMA = versione più stabile dei pesi nel tempo.

La EMA non viene utilizzata al posto del modello durante il normale backpropagation. Il modello continua infatti ad essere aggiornato normalmente dall'optimizer.

La copia EMA viene invece aggiornata dopo ogni:

optimizer.step().

Durante la validazione, il TrainerV2 sostituisce temporaneamente i pesi del modello con quelli EMA:

current weights $\rightarrow$ EMA weights $\rightarrow$ validation.

Terminata la validazione, i pesi originali vengono ripristinati.

In questo modo si può valutare il modello utilizzando una versione più smussata dei parametri senza alterare lo stato reale del training.

6.7 Gradient Clipping

Il TrainerV2 mantiene inoltre il *gradient clipping* già presente nel Trainer base.

Dopo il backward:

$$L \rightarrow \nabla_{\theta} L$$

viene limitata la norma complessiva dei gradienti.

Nel progetto:

```
torch.nn.utils.clip_grad_norm_(
    self.model.parameters(),
    max_norm=1.0
)
```

La norma dei gradienti viene quindi limitata a:

$$\|\nabla_{\theta} L\| \leq 1.$$

Se la norma supera il limite, i gradienti vengono ridimensionati in modo da rientrare nel valore massimo.

Questo è particolarmente utile quando il training produce gradienti molto grandi, perché impedisce che un singolo aggiornamento provochi una variazione eccessiva dei parametri.

Il gradient clipping non modifica direttamente la loss: modifica il *gradient* utilizzato dall'optimizer per aggiornare i parametri.

6.8 Training Loop

Il ciclo di training del TrainerV2 segue una sequenza ben definita per ogni batch.

Images $\rightarrow$ Forward $\rightarrow$ Targets $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Gradient Clipping $\rightarrow$ Optimizer Step $\rightarrow$ EMA Update.

In dettaglio, prima le immagini vengono inviate al modello:

$$x \rightarrow f_{\theta}(x).$$

Il modello produce quindi le prediction della detection head.

Successivamente il *Target Generator* costruisce i target a partire dalle bounding box del dataset.

Prediction e target vengono quindi passati alla funzione di loss:

$$L = L_{\text{cls}} + L_{\text{reg}} + L_{\text{ctr}}.$$

A questo punto viene eseguito il backward:

$$L \rightarrow \nabla_{\theta} L.$$

I gradienti vengono limitati tramite gradient clipping e infine l'optimizer aggiorna i pesi:

$$\theta_{t+1} = \text{Optimizer}(\theta_t, \nabla_{\theta} L).$$

Se EMA è abilitata, dopo l'aggiornamento viene eseguito anche:

$$\theta_t^{EMA} = \alpha \theta_{t-1}^{EMA} + (1 - \alpha) \theta_t.$$

Questo processo viene ripetuto per tutti i batch dell'epoca.

6.9 Validation

Al termine di ogni epoca il TrainerV2 esegue la validazione.

Durante questa fase:

$$\text{model.train()} \rightarrow \text{model.eval()}.$$

Se EMA è abilitata, i pesi EMA vengono applicati temporaneamente.

La pipeline diventa:

$$\text{Validation images} \rightarrow \text{Detector} \rightarrow \text{Post-processing} \rightarrow \text{Detections} \rightarrow \text{Metrics}.$$

Vengono quindi calcolate le metriche del dataset, tra cui AP e AR.

La metrica utilizzata come riferimento principale per selezionare il modello migliore è l'AP:

$$\text{best model} = \arg \max_{\text{epoch}} AP.$$

6.10 Checkpoint

Il TrainerV2 salva lo stato dell'addestramento in checkpoint.

Un checkpoint non contiene solamente i pesi del modello, ma può contenere anche:

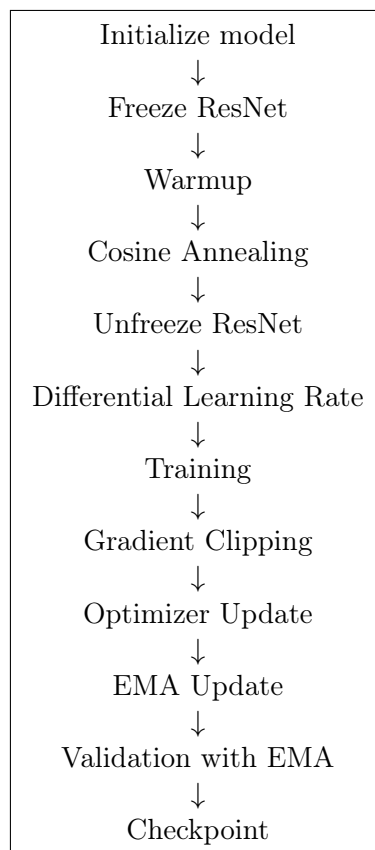
- stato dell'optimizer;
- stato del learning rate scheduler;
- epoca corrente;
- global step;
- migliore AP ottenuta;
- stato della EMA.

Questo permette di interrompere e successivamente riprendere il training mantenendo non solo i pesi del modello, ma anche lo stato dell'ottimizzazione.

Il TrainerV2 salva inoltre il modello con la migliore AP osservata e il checkpoint relativo all'ultimo stato del training.

6.11 Schema complessivo

Il comportamento del TrainerV2 può essere riassunto nel seguente schema:



Nel complesso, quindi, il TrainerV2 non introduce un nuovo detector, ma una strategia di ottimizzazione più sofisticata rispetto al Trainer base. Le scelte principali sono finalizzate a rendere più controllato e stabile l'adattamento di un backbone pretrained alle parti del detector specifiche del problema.